

Изучение аттракторов с помощью численных методов

Тодоров Николай

8 мая 2026 г.

Изучим систему нелинейных дифференциальных уравнений первого порядка

$$\begin{cases} \dot{x} = \sigma(y - x) \\ \dot{y} = x(r - z) - y \\ \dot{z} = xy - bz \end{cases} \quad (1)$$

Эта система называется системой Лоренца. Эту систему Эдвард Лоренц получил, когда изучал конвекцию в атмосфере Земли. Для нахождения ее численного решения мы воспользуемся методом Рунге-Кутты 4-го порядка

0	0	0	0	0
1/2	1/2	0	0	
1/2	0	1/2	0	0
1	0	0	1	0
	1/6	2/6	1/6	1/6

```
1  def solve(sigma, r, b, x0, y0, z0, number, T):
2      t = np.linspace(0, T, number)
3      h = t[1] - t[0]
4      sol = np.zeros((3, number))
5      sol[0, 0] = x0
6      sol[1, 0] = y0
7      sol[2, 0] = z0
8      for i in range(0, number - 1):
9          xi, yi, zi = sol[:, i]
10         k_1 = f(sigma, r, b, xi, yi, zi)
11         k_2 = f(sigma, r, b, xi + h * k_1[0]/2, yi + h * k_1[1]/2, zi + h *
12             k_1[2]/2)
13         k_3 = f(sigma, r, b, xi + h * k_2[0]/2, yi + h * k_2[1]/2, zi + h *
14             k_2[2]/2)
15         k_4 = f(sigma, r, b, xi + h * k_3[0], yi + h * k_3[1], zi + h *
16             k_3[2])
17         sol[:, i + 1] = sol[:, i] + h * (k_1 + 2 * k_2 + 2 * k_3 + k_4)/6
18     x = sol[0, :]
19     y = sol[1, :]
20     z = sol[2, :]
21     return t, x, y, z
```

У этого метода погрешность порядка $O(h^5)$ на каждом шаге и $O(h^4)$ суммарно на отрезке. Он является стандартом для изучения хаоса, так как сочетает высокую точность и относительно простую реализацию.

Эта система при $r \geq 1$ имеет три точки равновесия:

$$A = (0, 0, 0)$$

$$B = (\sqrt{b(r-1)}, \sqrt{b(r-1)}, r-1),$$

$$C = (-\sqrt{b(r-1)}, -\sqrt{b(r-1)}, r-1)$$

Нам будет интересовать решение при следующих значениях параметров:

$$\sigma = 10, r = 28, b = 8/3$$

Рассмотрим два решения с разными начальными условиями, отличающимися на малую величину (порядка 10^{-5}). Построим фазовый портрет обоих решений в 3D-пространстве, рядом отобразим зависимость расстояния (в смысле евклидовой нормы) между двумя решениями от времени. Как мы можем видеть, сначала точки движутся абсолютно одинаково, затем в какой-то момент времени они расходятся и живут отдельно друг от друга. Малая ошибка вначале экспоненциально возрастает. Это пример так называемого странного аттрактора. Формальное определение аттрактора следующее: это подмножество в фазовом пространстве, к которому стремятся фазовые кривые системы в течение продолжительного времени. Слово «странный» в данном контексте значит, что аттрактор имеет сложную, фрактальную структуру; траектории такого аттрактора не замыкаются, а малые отклонения очень быстро накапливаются (по экспоненциальному закону).

Это означает, что, **даже используя мощный компьютер, мы не можем просчитать траекторию, проходящую вблизи аттрактора, с разумной точностью на протяжении длительного промежутка времени. На каждом шаге вычислений неизбежно вносятся ошибки (из-за округления чисел и погрешностей численных методов), которые быстро накапливаются и приводят к тому, что найденная траектория сильно отличается от настоящей.**

Такое искажение невозможно исправить, просто увеличивая мощность компьютера. Подобное явление называется «динамическим хаосом».

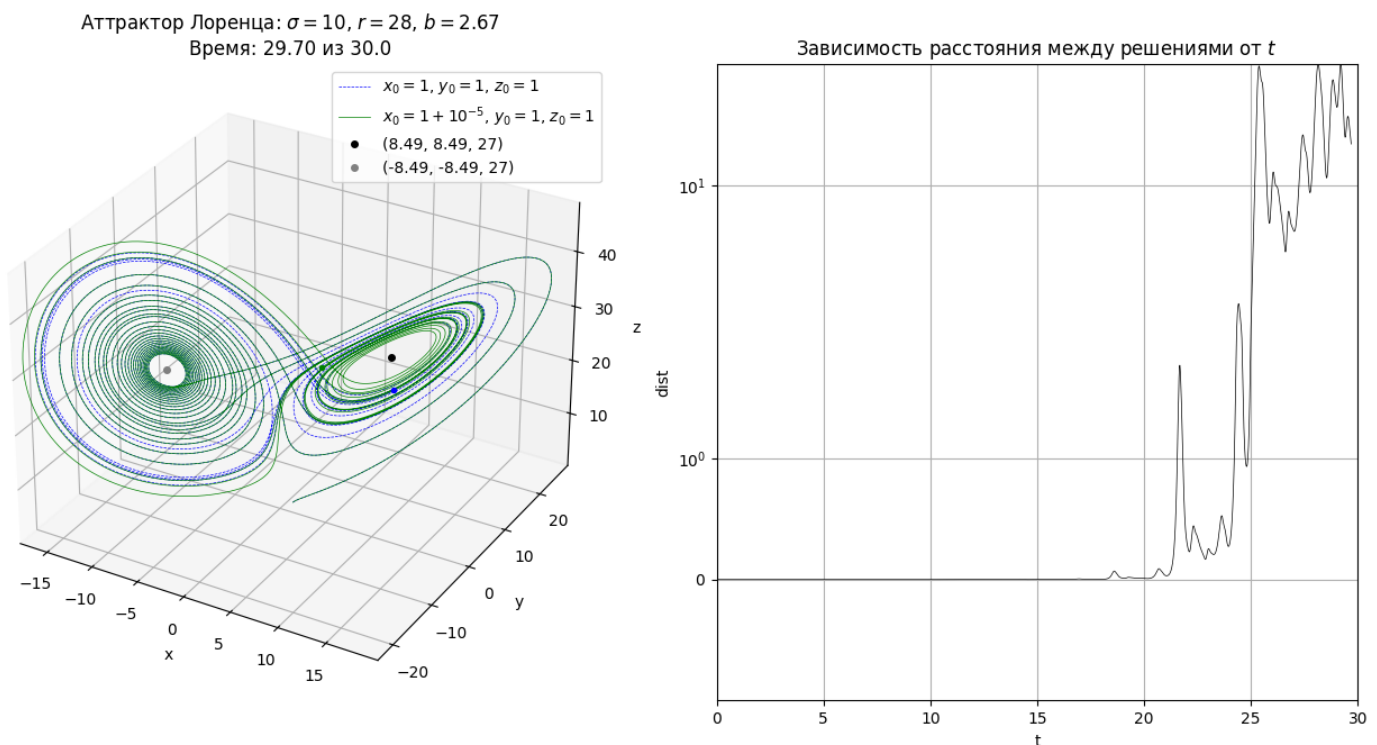


Рис. 1: Аттрактор Лоренца

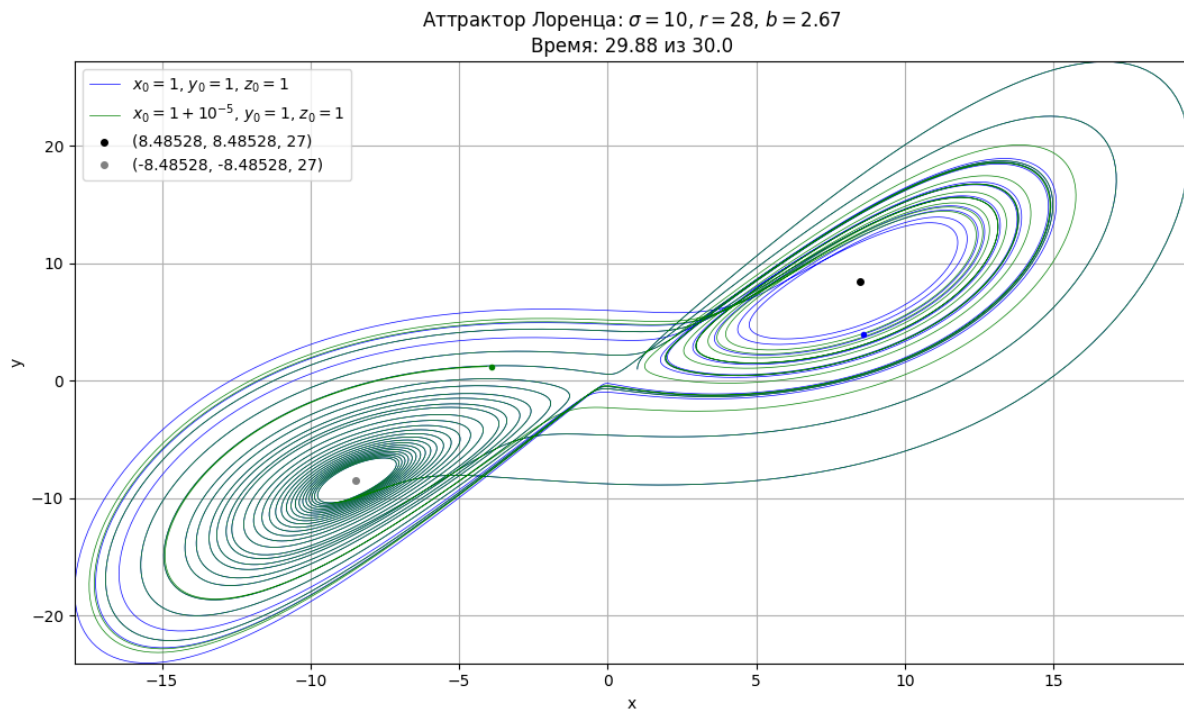


Рис. 2: Аттрактор Лоренца в XY

Аттрактор внешне похож на бабочку; каждое из ее крыльев имеет фокус B и C соответственно.

Покажем как выглядят графики зависимостей координат от времени. Выполним также преобразование Фурье от них (используя FFT с помощью библиотечной функции в numpy) и посмотрим на спектр частот (он будет достаточно широкий)

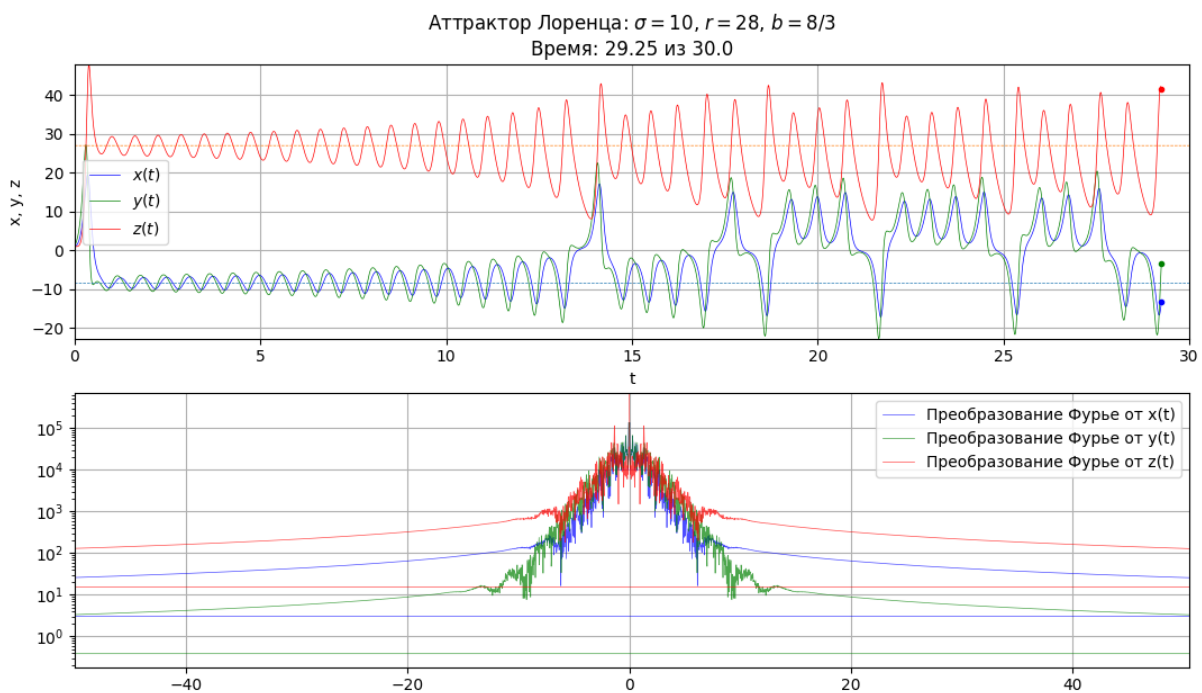


Рис. 3: Временные зависимости координат аттрактора Лоренца и их фурье-спектры

Теперь посмотрим, как выглядит бифуркационная диаграмма данной системы. В качестве регулирую-

шего параметра мы возьмем r , который будет изменяться от 0.01 до 30. По вертикальной оси мы будем откладывать локальные максимумы координаты x . Процесс нахождения этих максимумов описывается ниже написанным кодом

```

1  for r in r_vals:
2      t, x, y, z = solve(sigma, r, b, x_0, y_0, z_0, number, T)
3      x = x[2500:]
4      x_r_max = []
5      for i in range(1, len(x) - 1):
6          if x[i - 1] < x[i] > x[i + 1]:
7              x_r_max.append(x[i])
8              x_max.append(x_r_max)

```

Разбить полученную диаграмму можно на три участка: на первом - максимумы расположены выше нуля, на втором - ниже нуля, на третьем - как выше нуля, так и ниже нуля. Концептуально это означает следующее: при малых значениях параметра r фазовые кривые притягиваются к точке равновесия B , при средних - к точке C ; при достаточно больших значениях этого параметра решение не стремится к какому-либо фокусу, оно периодически сменяет крыло и не затухает в отличие от решений при малых r .

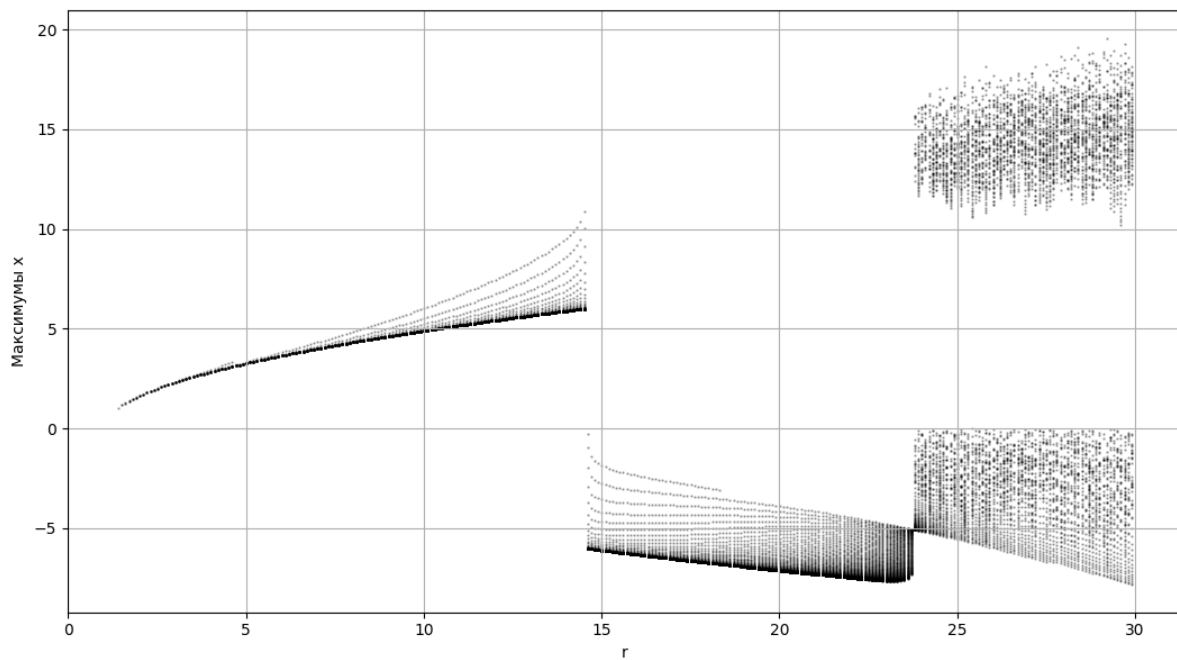


Рис. 4: Бифуркационная диаграмма